CMF 3D-QSAR

The continuous molecular fields approach, as a program

Desktop application, library and command-line tool
for the Continuous Molecular Fields approach to 3D-QSAR
User manual

Contents

What this program is **4**

- The idea in one page 4
- What you can do with it 4
- How this manual is arranged 5

Installation and first run **6**

- Requirements 6
- Starting the application 6
- A first model in five minutes 6

The application window **7**

- The data-set panel 7
- The model panel 7
- The 3D view and its controls 7
- The results panel 8
- The menus 8

Preparing the data **10**

- What the program reads 10
- Activities 10
- Alignment 10
- Charges and per-atom properties 11

Building a model **12**

- Choosing the fields 12
- Attenuation, regularisation and the grids 12
- The learners 12
- Nonlinear kernel layers and multiple kernel learning 13
- Validation 13
- Statistics reported 13
- Saving and reusing 14

Reading the model **15**

- Co-fields 15
- Contributions 15
- Shapley values 15
- A pharmacophore out of the co-field 16
- What a candidate is missing 16
- Chemical space maps 16

Classification and virtual screening **17**

- Setting up a classification task 17
- One-class models: screening with actives alone 17
- Decoys and the figures that matter 17

Conformations: the fourth dimension 19

Working without an alignment 20

- Why the frame is the problem 20
- The pose-free count kernel 20
- The rotation-invariant autocorrelation kernel 20
- Topomer fields 21
- What the three routes are worth, measured 21
- Heterogeneous series, step by step 22

Protein pockets 23

- Getting a pocket on screen 23
- Complementarity as numbers 23
- Docking 23

Command-line reference 24

- The options of `build`, by group 24
- Worked examples 25

Using it as a library 26

The example data sets 27

When something goes wrong 28

Appendix A — Field parameterisation 29

Appendix B — Files and tags the program understands 30

Appendix C — The papers 31

What this program is

CMF 3D-QSAR is a desktop application, a Python library and a command-line tool implementing the **Continuous Molecular Fields** approach to three-dimensional structure–activity modelling. The method was introduced by Zhokhova, Baskin and co-workers (Dokl. Chem. **429** (2009) 273; J. Comput.-Aided Mol. Des. **27** (2013) 427; continuous indicator fields: J. Comput.-Aided Mol. Des. **29** (2015) 233; co-fields: Future Med. Chem. **11** (2019) 2701), and the program follows those papers closely enough that their published statistics can be reproduced from the same data — `docs/benchmark.md` records that comparison set by set.

The idea in one page

A molecule is represented not by a table of descriptors and not by values on a lattice, but by **continuous functions of space**. Each field is a sum of Gaussians centred on the atoms,

```
X_f(r) = Σ_l w_lf exp(−α_f |r − r_l|² / 2)
```

where `w_lf` is the weight of atom *l* in field *f* — its partial charge for the electrostatic field, its Crippen logP contribution for the hydrophobic field, 1 for an indicator field of a given atom type, or any number the file carries per atom — and `α_f` is the attenuation factor that sets how far the field reaches.

Two molecules are compared by the **overlap integral** of their fields, which for Gaussians has a closed form:

```
K_f(A, B) = ∫ X_fA(r) X_fB(r) dr = (π/α)^{3/2} Σ_l Σ_m w_lf w_mf exp(−α |r_l − r_m|² / 4)
```

No grid is ever evaluated for this: the kernel is a sum over atom pairs, computed exactly. The kernels of the individual fields are mixed with coefficients `h_f` into one kernel, and that kernel goes into a kernel method — ridge regression, a support vector machine, a Gaussian process. Three consequences follow, and they are what distinguishes the approach from CoMFA:

- **Nothing is discretised.** There is no lattice spacing to choose, no alignment of grids, no variable selection among tens of thousands of columns, and no truncation at a cut-off distance.
- **The model is a function of space again.** A kernel ridge model predicts `y(X) = Σ_j a_j K(X, X_j) + b`, and because the kernel is an integral of a product, the whole sum can be rewritten as the overlap of the molecule's field with one function, the **co-field** `C_f(r) = Σ_j a_j X_fj(r)`. The co-field is the program's map: it plays the role of CoMFA's coefficient contour, but it is the model rather than a projection of it.
- **The decomposition is exact.** Because the prediction is bilinear in the fields, it can be split without approximation into contributions of training molecules, of field types, of regions of space, or of individual atoms.

What you can do with it

Task	How
Build a 3D-QSAR regression model on an aligned congeneric series	Model panel, or `cmf_cli.py build`
Work without an alignment	topomer fields, the rotation-invariant autocorrelation kernel, or the pose-free count kernel
Classify actives against inactives, or screen with actives alone	binary (SVC, ν-SVC, Laplace GP) and one-class (1-SVM, centroid, k-NN, GP, kernel PCA) tasks
See what the model says about space	co-fields as isosurfaces, molecular fields, overlap maps, Gaussian-cube export
See what the model says about a molecule	training-molecule contributions, atom contributions, Shapley values and interaction indices
Turn a model into design instructions	pharmacophore extraction from co-field lobes, field-difference maps against a reference ligand
Use the protein	fetch or load a pocket, draw it behind the fields, score field complementarity, dock
Average over conformations	conformational ensembles (4D) and multi-instance learning
Map a data set	kernel GTM chemical-space maps with activity, class and density landscapes

How this manual is arranged

Chapters 2 and 3 get you running and name every part of the window. Chapters 4 to 6 are the working cycle: prepare the data, fit a model, read it. Chapters 7 to 10 are the specialised routes — screening, conformations, alignment-free modelling, protein pockets. Chapter 11 is the complete command-line reference, chapter 12 the library, chapter 13 the example data, and chapter 14 collects the things that go wrong. The appendices hold the field parameterisation, the file conventions and the bibliography.

Conventions: *Menu → Item* names a menu command, `monospace` names a file, an option or a piece of code. Equations are quoted in the notation of the papers, with their equation numbers.

Installation and first run

Requirements

Python 3.10 or newer, and the packages in `requirements.txt`:

```
pip install -r requirements.txt
```

That brings RDKit (chemistry), PySide6 (the interface), pyqtgraph with PyOpenGL (the 3D view), scikit-image (marching cubes for the isosurfaces), scikit-learn (the SVM solvers), NumPy, SciPy and joblib. The program is pure Python and needs no compilation step; it has been used on Windows, Linux and macOS.

Optional extras, only needed for particular features:

- `dimorphite_dl` — protonation states when importing SMILES for a given pH;
- `matplotlib` — supplies the DejaVu fonts used to typeset this manual;
- a local ORCA or VASP installation if you intend to compute per-atom quantum properties to use as fields (see *Preparing the data* and appendix A).

Starting the application

```
python cmf_app.py
```

or straight onto a data set, which saves two dialogs:

```
python cmf_app.py examples/benzamides_synthetic.sdf --activity pActivity --test-fraction 0.25
```

If the 3D view stays black, the OpenGL driver is the usual cause; see *When something goes wrong*.

A first model in five minutes

1. *File → Open training set…* and choose `examples/benzamides_synthetic.sdf`. It holds 109 aligned N-phenylbenzamides with a synthetic activity, so you know the answer the model should find. Pick `pActivity` as the activity property.
2. In the data-set table, press *Random split* and set a quarter of the molecules aside as a test set.
3. In the Model panel leave the five physico-chemical fields ticked, leave the method at kernel ridge regression, and press **Build model**.
4. Read the *Statistics* tab: q^2 should be about 0.87 and R^2_{pred} about 0.84 — the noise ceiling of that set is 0.94, so the model has recovered nearly all of the signal that was put into it.
5. Press **Compute co-field**, choose *Electrostatic*, and move the *iso level* slider. The red and blue lobes that appear are the pseudo-receptor zones the activity was generated from.

Nothing in that sequence is specific to the example: it is the working cycle of the program.

The application window

The window has four regions: the **data-set panel** on the left, the **model panel** below or beside it, the **3D view** in the middle with its controls above it, and the **results panel** with its tabs on the right. *View → Restore default layout* puts everything back if a panel gets lost, and *View → Results panel spans full width* gives the tables and plots the whole width when you are reading rather than building.

The data-set panel

One row per molecule: name, activity, subset (train, test or decoy), the fitted and cross-validated predictions once a model exists, and the chemotype once the chemotypes have been found. The panel is where molecules are selected, shown and hidden.

- **Clicking a column header sorts by it** — activity, name, subset, either prediction column or the chemotype. A second click reverses the order and a third restores the order of the file. Molecules with no value in that column always sink to the bottom. The selection, the *Show* ticks and everything else follow the molecule rather than the row, so sorting never shuffles what is on screen.
- The *Show* tick controls what the 3D view draws. *Show all* and *Show none* are the shortcuts; with one molecule shown, the co-field and the atom contributions refer to it.
- *Random split* and the → *test* button define a test set; a `subset` tag in the file is honoured automatically, so published splits arrive with the data.
- A note under the table records a transformation that has been applied to the data set, for instance *[topomer substituents of …]*, so that a changed data set never looks like the original.

The model panel

Everything that defines a model: the fields and their parameterisation, the task, the learner, the hyper-parameter grids, the validation scheme. Chapter 5 goes through it control by control. Two buttons act: **Build model** runs the grid search, the final fit, the internal cross-validation and the test prediction; **External CV** runs the nested procedure that reports q^2ex.

The 3D view and its controls

Mouse: drag to rotate, wheel to zoom, middle-drag (or shift-drag) to pan. The controls above the view are arranged in rows — molecule display, maps, iso level, protein site — and all of them redraw immediately:

Control	Effect
Style	ball & stick, sticks, lines, space fill
Labels	none, element + number, atom type, partial charge
Maps	co-field, molecular field, overlap, atom contributions, Shapley field, difference map, pharmacophore
Field	which field type the maps refer to
Iso level	the isosurface threshold, as a percentage of the largest absolute value

Control	Effect
Style per map	solid or mesh, chosen separately for co-fields, molecular fields and Shapley fields
Protein site	distance cut-off, wire or sticks, hydrogens, waters, residue labels

Co-field colours are fixed so that maps from different sessions can be compared: electrostatic red (+) / blue (−), steric green (+) / yellow (−), hydrophobic orange (+) / light blue (−), H-bond acceptor magenta (+) / dark red (−), H-bond donor cyan (+) / purple (−), and indicator fields red (preferable position of that atom type) / blue (undesirable).

Tools → Center view on current molecule recentres the camera, and *File → Save 3D view as image...* writes what you see as a PNG.

The results panel

Tab	What it holds
Statistics	every figure of merit for the fit, the cross-validation, the test set and the external CV
Scatter plot	observed against predicted, by subset, with the molecule under the cursor identified
ROC	receiver-operating curves for a classification model, per subset
Scores	decision values or calibrated probabilities, sortable, for a classification or screening model
Enrichment	the enrichment curve against a decoy set, with EF at 0.5, 1, 5 and 10 %, BEDROC and RIE
Map	the kernel-GTM chemical-space map with its activity, class and density landscapes
Hyper-parameter search	the whole grid, so you can see whether the optimum is interior or at an edge
Contributions	which training molecules drive the selected molecule's prediction, and the atom contributions
Shapley	Shapley values over atoms, substituents or field types, with second-order interactions
Complementarity	field complementarity of molecules and co-fields with the protein pocket
Chemotypes	the Murcko scaffolds found in the set and the leave-one-chemotype-out result
Log	everything the program did, with timings; the first place to look when a result surprises you

The menus

File — open a training set, add a test set, add decoys, import activities from CSV; load or fetch a protein structure, or forget it; load and save models; export predictions (CSV), the aligned data set (SDF), the current co-field or Shapley field (Gaussian cube) and the 3D view (PNG).

View — show or hide the panels, let the results panel span the full width, restore the default layout.

Tools — generate conformations (4D); dock into the pocket; align the data set to a reference; align by prediction confidence, for a test set or cross-validated for the training set; replace the molecules by their topomer substituents; field difference from a reference molecule; extract a pharmacophore from the co-field; chemical-space map (GTM); align decoys to the training set; unalign (randomise poses); centre the view; and the four-step *Heterogeneous sets* submenu.

Help — a method summary with the equation numbers of the papers.

Preparing the data

What the program reads

Format	Notes
`.sdf`, `.sd`, `.mol`	3D coordinates used as they are; hydrogens are added with coordinates if missing
`.mol2`	Sybyl atom types and partial charges are read and can be used directly; a tolerant fallback reader handles blocks RDKit refuses
`.smi`, `.csv`, `.tsv`, `.txt`	2D input: each molecule is embedded with ETKDG and minimised with MMFF94, which means **the poses are arbitrary** until you align them
`.pdb`, `.mmcif`	protein structures only, read by a deliberately permissive reader

A CSV needs a column of SMILES; a column called `name` or the first non-SMILES column names the molecules, and every other column is kept as a molecule property, so activities and metadata travel with the file.

Activities

Three ways, in order of convenience:

1. **An SD property.** The loader guesses the activity property and the dialog lets you change it. Anything numeric will do: `pIC50`, `pKi`, `pActivity`, `CBG_pK`.
2. **A separate table** (*File → Import activities from CSV…*, `--activity-file`): molecule name, activity, and optionally a third column with the split (1 = train, 2 = test, or the words).
3. **A class property** for classification tasks, or a threshold on the activity.

A per-molecule tag named `subset`, `set` or `split` is honoured automatically, which is how the published splits of the example sets arrive.

Alignment

CMF compares fields in one frame of reference. If your molecules came from a superposition, from a crystallographic series or from a docking run, that frame already exists and you should leave it alone. If they came from SMILES, they have no frame at all and you must make one — or use one of the frame-free routes of *Working without an alignment*, which is often the better answer.

The tools, in increasing order of sophistication:

- *Tools → Align data set to reference…* — rigid superposition on the maximum common substructure with a reference molecule, optionally refined by maximising the CMF kernel similarity, which is the alignment criterion proposed in the 2013 paper.
- *Tools → Align by prediction confidence…* — minimises the Gaussian-process posterior variance of a molecule under an existing model, which moves it into the model's applicability domain. It uses no activity value, so it is valid for prospective prediction. The program refuses to move training molecules, because the optimum there is degenerate: superimposing everything maximises mutual similarity and destroys the signal.

- *Tools → Align decoys to the training set…* — the same idea for a background set, so that no decoy is rejected merely for having been posed badly.
- *Tools → Unalign (randomise poses)…* — the control experiment. It gives every molecule a random rigid pose, which leaves internal geometry and self-kernels untouched and scrambles only the mutual arrangement. Randomise, realign, and see how much comes back; the program remembers the poses from before and reports the recovery.

Charges and per-atom properties

The electrostatic field needs charges: Gasteiger (the default), MMFF94, or **read from the file**, which is what you want with the Sutherland sets and their MNDO ESP-fit charges, or with anything you computed yourself.

Beyond that, any per-atom number can become a field of its own. `prop:<name>` reads the atomic weights from a property of the molecule — a quantum charge, a Fukui index, a local softness, an atomic polarisability, even a per-atom embedding — and that field then behaves exactly like a built-in one, with its own attenuation, its own mixing coefficient, its own co-field, atom contributions and Shapley values.

Two conventions are read, in this order:

1. a property set on **every atom**, which RDKit writes to SDF as `atom.dprop.<name>` and reads back automatically;
2. a **molecule property holding one number per atom**, separated by spaces, commas or newlines, which is how most quantum-chemistry packages dump such a table.

The hydrogens are the trap. A file without 3D coordinates is embedded on loading and hydrogens are added, so a list written for the heavy atoms alone no longer matches the atom count. The program says so rather than silently mis-assigning: `--prop-missing error` (the default) stops, `--prop-missing zero` gives the unlisted atoms zero weight. `examples/orca_atom_properties.py` shows the whole route from an ORCA run to an SDF the program reads.

Building a model

Choosing the fields

The five physico-chemical fields — electrostatic, steric, hydrophobic, H-bond acceptor, H-bond donor — are the default and reproduce the published CMF models. The steric field is special: its width is scaled by the van der Waals radius of each atom (`α/R²_vdW`, Eq. 10–11 of the 2013 paper) rather than being shared.

Continuous indicator fields (CIF, 2015 paper) add one field per atom type, with weight 1 on atoms of that type and 0 elsewhere. Tick *Continuous indicator fields* and set the minimum number of training molecules an atom type must appear in (3 by default) to be given a field of its own. Three typing schemes are available: `sybyl` (the types of the original scripts, read from MOL2 or assigned), `element`, and `element+aromatic`. CIFs are the most descriptive field set the program has, and on several of the benchmark sets they match or beat the physico-chemical fields on their own.

`prop:<name>` fields are added by naming them in the per-atom property row of the model panel, or in `--fields`.

Attenuation, regularisation and the grids

Two hyper-parameters matter. **α** sets how far each field reaches: large α makes the fields sharp, so molecules must match closely, while small α makes them diffuse. **λ** (or C, ε, ν for the SVM learners) sets the regularisation. Both are chosen by cross-validated grid search, and the whole grid is shown in the *Hyper-parameter search* tab — look at it, because an optimum at the edge of the grid means the grid was too small.

```
python cmf_cli.py build data.sdf --alphas 0.1,0.3,0.5,1,2,5 --lams 1e-3,1e-2,0.1,1,10
```

Refinements, all optional: continuous Brent minimisation over λ as in the original R scripts (`--lam-search brent`), a Nelder–Mead search over α (`--alpha-search nelder-mead`), coordinate descent or centred kernel alignment for the mixing coefficients `h_f` (`--optimize-h`, `--mkl`), and a grid over the bandwidth of a nonlinear kernel layer (`--gammas`).

Two conventions decide how the field kernels are combined, and they change the numbers: *Normalise kernels* uses cosine-normalised field kernels, and *Scale fields* rescales each field kernel to unit mean diagonal before mixing. Untick the second to get the raw-kernel convention of the papers (`--raw-scale`).

The learners

Task	Methods	Key
Regression	kernel ridge regression with bias; Gaussian process (the same fit plus predictive variances); ε-SVR; ν-SVR	`krr`, `gp`, `svr`, `nusvr`
Binary classification	C-SVC; ν-SVC; Gaussian-process classifier by the Laplace approximation	`svc`, `nusvc`, `gpc`

Task	Methods	Key
One-class / screening	1-SVM; distance to the centre of the actives; distance to the k-th nearest active; one-class Gaussian process; distance from the principal subspace (kernel PCA)	`ocsvm`, `ocmean`, `ocknn`, `ocgp`, `ockpca`

Kernel ridge regression is the method of the papers and the default. The Gaussian-process view adds a posterior variance per prediction at no extra cost, which is what the confidence-based alignment of *Preparing the data* minimises. The Laplace GP classifier adds calibrated probabilities, a posterior variance and model selection by marginal likelihood — it scores no better than C-SVC on the benchmark sets (mean difference 0.001 in Matthews correlation), so it is not the default, but its probabilities are honest and its evidence picks hyper-parameters without holding anything out.

Nonlinear kernel layers and multiple kernel learning

A layer on top of the CMF kernel makes the model more expressive than an overlap integral: a Gaussian layer, $\exp(-\gamma d^2)$ with $d^2 = K(i,i) + K(j,j) - 2K(i,j)$, or one or two arc-cosine layers, the kernel of an infinitely wide neural network (Cho & Saul 2009). The price is interpretation: the model is then no longer linear in the field, so co-fields and atom contributions are unavailable. The similarity decomposition stays valid, and the **Shapley tab** decomposes the prediction instead, which is why it exists.

Multiple kernel learning chooses the mixing coefficients `h_f` rather than leaving them at 1, either by coordinate descent on the cross-validated error or by centred kernel alignment (`--mkl align|alignf`), which is a closed-form, training-only criterion.

Validation

- **k-fold cross-validation** (10 by default) gives q^2 and RMSEcv. Folds are shuffled unless you ask for the interleaved folds of the original scripts (`--cv-no-shuffle`).
- **A test set** gives R^2pred and RMSEpred. Both variance conventions are reported, because they differ for sets whose test activities span a narrow range — the steroid benchmark is the classic example, where the ten test affinities cover 1.9 log units and R^2pred computed against the test variance is negative while the predictions are good.
- **External (nested) cross-validation** (`--nested`, *External CV*) wraps the whole model selection in an outer loop and reports q^2ex and RMSEcvex. This is the number to quote: the ordinary q^2 has seen the hyper-parameter choice.
- **Leave-one-chemotype-out** folds, for a set of several scaffolds, never leave a molecule's own scaffold-mates in the training part. Random folds flatter such sets badly — by 0.11 to 0.47 q^2 on the five chemotype benchmarks here — and the program reports both, together with an alignment-free fragment baseline that the aligned model has to beat.

Statistics reported

Regression: r^2 and RMSE for the fit, q^2 and RMSEcv for the cross-validation, R^2pred and RMSEpred for the test set in both variance conventions, q^2ex and RMSEcvex for the external CV. Classification: accuracy, balanced accuracy, sensitivity, specificity, Matthews correlation, AUC and the ROC curve, each for the fit, the cross-validation and the test set. Screening: AUC, enrichment factors at 0.5, 1, 5

and 10 %, the best achievable enrichment at each of those fractions, BEDROC and RIE.

Saving and reusing

File → Save model… writes a `.cmf` file (joblib) holding the field set, the kernels' definition, the fitted coefficients, the training molecules and every setting needed to predict later. `cmf_cli.py predict model.cmf new.sdf --out predictions.csv` applies it; the GUI loads it with *File → Load model…*.

Reading the model

A CMF model is readable in a way a grid model is not, because the prediction can be taken apart exactly. The 2019 paper states three decomposition rules, and the program implements all three.

Co-fields

The co-field is the model as a function of space:

```
C_f(r) = Σ_j a_j X_fj(r)
```

a weighted sum of the training molecules' fields with the fitted dual coefficients as weights. The prediction of any molecule is then the overlap of its own field with the co-field, field type by field type. Press **Compute co-field** and the program evaluates it on a grid around the training set; choose the field with the *Field* selector and set the *iso level* slider, which is a percentage of the largest absolute value rather than an absolute number, so the surfaces stay meaningful when you change field.

What the lobes mean: a positive co-field lobe is a region where *more* of that field raises the predicted activity. For the electrostatic field that means positive charge is favoured there, for the steric field bulk, for the hydrophobic field lipophilicity, and for an indicator field it means an atom of that type is welcome at that position.

Two further maps use the same grid, for the molecule currently shown:

- the **molecular field** `X_f(r)` of that molecule, in the same colours, drawn as a mesh by default so that it can be seen together with the co-field;
- the **overlap** `C_f(r) X_f(r)`, whose integral *is* that field's contribution to the prediction (Eq. 7), so the map shows where the prediction comes from.

File → Export current co-field (Gaussian cube) writes a cube file readable by PyMOL, VMD or Jmol, which is the way to put a co-field into a publication figure.

Contributions

The *Contributions* tab answers two questions about the selected molecule:

- **Which training molecules** make its prediction, as `a_j K(X, X_j)` — the similarity decomposition, rule 2. A prediction resting on one or two training molecules is a warning.
- **Which of its atoms**, as the atom contributions of Eq. 14 of the 2015 paper, drawn in the 3D view as red and blue spheres whose size is the magnitude. This is the most directly actionable map the program produces: it says *this* substituent is helping and *that* one is not.

Shapley values

The *Shapley* tab decomposes a single prediction into additive contributions by the Shapley construction, over **atoms**, over **substituents of the common scaffold**, or over **field types**, with second-order interaction indices available. It serves two purposes. With the linear kernel it reproduces the atom contributions exactly, which is a useful check. With a nonlinear kernel layer, where co-fields do not exist, it is the only decomposition left — and it still works, because it needs nothing but the ability to evaluate the model on subsets.

A pharmacophore out of the co-field

Tools → Extract pharmacophore from the co-field... turns the co-field into a discrete, typed pharmacophore: the favourable lobes of each field become feature points of the matching kind (positive and negative centres, hydrophobe, donor, acceptor), each with a tolerance radius taken from the size of its lobe, and the unfavourable steric lobes — or the pocket, when a protein is loaded — become exclusion volumes. The result is a table and a PDB-style file of feature spheres that other software can read, and it draws in the 3D view; untick *pharmacophore* in the viewer controls to clear it.

This is the bridge from a CMF model to a screening query: the model is continuous and needs an alignment, the pharmacophore is discrete and does not.

What a candidate is missing

Tools → Field difference from a reference molecule... subtracts two molecules' fields on the co-field grid and reports, per field type, the cosine between them, the distance, and how much of the reference's field the candidate **fails to reproduce** against how much it **adds**. The difference map draws as its own isosurface, and the table ranks the whole data set by field similarity to the reference. In a lead-optimisation reading: the "missing" column is what to put back, the "added" column is what may be causing trouble.

Chemical space maps

Tools → Chemical space map (GTM)... builds a two-dimensional map of the set from the CMF kernels. Fields are not vectors, so Generative Topographic Mapping cannot be applied to them directly; the program performs kernel PCA on the CMF kernel — which is functional PCA, and reproduces feature-space distances exactly — and runs the ordinary EM fit of GTM in those coordinates. The out-of-sample projection stays a single kernel row, so new molecules can be placed on an existing map.

From the responsibilities follow the **activity landscape** (responsibility-weighted mean activity per node), the **class landscape** (the fraction of actives) and the **data density**, which is a picture of the applicability domain; nodes with no support are left blank. The landscape is a model in its own right: on the thrombin training set it reproduces the activities with R^2 0.92.

Two cautions. Use cosine-normalised kernels, or the leading components follow molecular size. And a CMF kernel between molecules that share no frame is meaningless, so a map spanning several series would partly be showing alignment quality — unless you use the pose-free count kernel of *Working without an alignment*, which separates three targets' ligands cleanly with no alignment at all.

Classification and virtual screening

Setting up a classification task

Choose *Binary classification* as the task. The classes come either from a threshold on the activity (with *active below* for data where small is potent, such as an IC50 in molar units) or from a class property in the file. The learners are C-SVC, ν-SVC and the Laplace Gaussian-process classifier; the *ROC* and *Scores* tabs replace the scatter plot.

```
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file \
       --task classification --threshold 6.84 --method gpc --Cs 1,10,100
```

One-class models: screening with actives alone

Often there are no reliable inactives, only a set of known actives, and the question is whether a new molecule belongs to the same class. That is one-class classification, applied to CMF by Karpov et al. (2011). Five descriptions of the active class are implemented:

Method	What it measures
`ocsvm`	the 1-SVM: a hyperplane in feature space separating the actives from the origin
`ocmean`	distance to the mean embedding of the actives, the centroid of the fields
`ocknn`	distance to the k-th nearest active
`ocgp`	the one-class Gaussian process, optionally scored by mean over standard deviation
`ockpca`	reconstruction error against the principal subspace of the actives (kernel PCA)

On the sets measured here the 1-SVM is the **most fragile** of the five: the centroid, k-NN and kernel-PCA descriptions are as good or better and have one parameter fewer to go wrong. Try more than one; they disagree in interesting places.

Decoys and the figures that matter

A screening model is only as meaningful as the background it is tested against. *File → Add decoys (background set)…* or `--decoys` loads presumed-inactive molecules used **only** for evaluation, never for fitting. `examples/make_decoy_sets.py` builds property-matched decoy sets, which is the honest kind: decoys matched to the actives in molecular weight, logP, rotatable bonds and formal charge, so that the model cannot win by noticing size.

Two traps the program handles explicitly:

- **A decoy must not be rejected for its pose.** `--align-decoys` places every decoy in the frame of the training set by maximising its field similarity, with several random restarts, and `--decoy-conformers N` generates N conformations of each decoy and keeps the one that poses best (the file's own conformation is always among the candidates).

- **AUC saturates.** With cross-target decoys almost any method reaches 0.95 and above, which tells you nothing. Read the **enrichment factors** at 0.5, 1, 5 and 10 % instead, which the program reports beside the maximum achievable value at each fraction, and **BEDROC/RIE** (Truchon & Bayly), which weight the top of the list.

A measured caution from the benchmarks in this repository: field similarity on these tasks performs about as well as ECFP4 fingerprint similarity, and cross-target decoys are trivially separable by either. If a screening result looks spectacular, check what the decoys were.

Conformations: the fourth dimension

A 3D-QSAR model represents each molecule by one conformation, and the answer depends on which one. 4D-QSAR (Hopfinger et al. 1997) replaces the choice by an ensemble and averages over it. *Model → Conformations* switches between the two; **single is the default and is untouched**, and the tests assert that by comparing its fields against those built before ensembles existed.

In CMF the averaging needs no new mathematics. A field is a sum of atom-centred Gaussians and the kernel is bilinear in the fields, so the mean field of an ensemble is itself a field — its atoms are the atoms of every conformation with their weights multiplied by that conformation's weight — and the kernel of two ensembles comes out as

```
K(A, B) = Σ_k Σ_l w_k^A w_l^B K(X_A^(k), X_B^(l))
```

the mean-map kernel of the two conformational distributions, positive definite by construction. A test checks that identity against the pairwise kernels of the conformations. The model stays linear in that mean field, so **co-fields, the decomposition rules, Shapley values and the complementarity scores all keep working**; contributions arrive per atom per conformation and fold back onto the atoms by summation.

Tools → Generate conformations... builds the ensembles with ETKDG and MMFF94s, keeping the pose a molecule already has as its first conformation, so an earlier alignment survives. A multi-conformer file can be read instead, with `--ensemble-by name` grouping the records of one compound. Weights are uniform or Boltzmann from the stored energies, with kT and a cap on the number kept.

Where the other conformations go is a choice of its own, and not a small one: what matters is not where a conformation sits relative to its own molecule but where it sits relative to the rest of the series.

`--conformer-align`	**Where each generated conformation is put**
`self`	on the molecule's own pose, over all heavy atoms — the default, so nothing changes silently
`core`	on its own pose, over the common substructure with a reference only, so the scaffold stays where the published alignment put it and only the flexible part moves
`reference`	on a reference molecule of the series, through that same common substructure, which puts every conformation of the series in one frame
`field`	as `reference`, then refined by maximising the normalised CMF kernel — the 2013 paper's criterion, applied per conformation

Multi-instance learning (`cmf_cli.py mil`) goes further: instead of averaging, the model chooses which conformation of each molecule to use, alternating between fitting and selecting. Use it when you believe one conformation is the bioactive one and the others are noise; use the ensemble when you believe the molecule is genuinely flexible.

Working without an alignment

This chapter is the one to read if your molecules came from SMILES, if your series has several scaffolds, or if you simply do not trust the superposition you have. The program offers three frame-free routes, and the scripts in `benchmarks/` measure all of them.

Why the frame is the problem

An alignment is an assumption, and a wrong one is expensive. Measured on the nine targets that occur both in the curated Sutherland collection and in the automatically aligned Mittal collection — same compounds, same activities, same fields and grids — the curated superposition gives a mean q^2 of **+0.597** and the automatic one **+0.295**. Worse, the gain over a baseline with no geometry at all (atom-type counts) is +0.25 on the curated files and **−0.06** on the automatic ones: on five of the nine targets, fields in an automatic frame predict *worse* than counts. An alignment that is not trustworthy is not merely imperfect, it is actively harmful.

The pose-free count kernel

As $\alpha \to 0$ an indicator field tends to the count of atoms of that type, so the kernel tends to the dot product of the atom-type count vectors and nothing of the geometry survives. The program implements that limit in closed form (`chemotypes.count_kernel`), which is exact rather than approached, costs one pass over the atoms, and is 10 to 34 times faster than the Gaussian route. It is the **baseline** any aligned model should be made to beat, and it is what the leave-one-chemotype-out workflow reports beside the aligned model.

One subtlety, because it is easy to get wrong: the normalisation must be applied to the joint kernel, not field by field. A field normalised on its own tends to 1 whenever both molecules contain the type at all, so the counts cancel and the baseline loses the very descriptors it is built from.

The rotation-invariant autocorrelation kernel

Geometry without a frame. For each pair of field types the program computes the smoothed field autocorrelation

$$\Phi_{fg}(d) = \Sigma_l \Sigma_m w_{lf} w_{mg} \exp(-(|r_l - r_m| - d)^2 / 2\sigma^2)$$

as a function of interatomic distance d, which is invariant under rotation and translation by construction, and takes the linear kernel on those channels. The $d = 0$ channel reproduces the CMF self-overlap exactly when $\sigma^2 = 2/\alpha$, so the descriptor contains the ordinary kernel's diagonal information and adds the distance profile. Select *Autocorrelation* as the kernel source in the model panel (`--kernel autocorrelation`, with `--ac-sigma` and `--ac-dmax`), and the model fits, cross-validates, saves and predicts exactly as any other — only the co-field, which needs a frame, is unavailable.

On the chemotype benchmarks the invariant kernel recovers most of the value of the geometry without a frame (0.172 against 0.189 for the aligned model and 0.072 for counts), and on automatically aligned files it frequently beats the aligned model outright — on six of the nine shared targets, including thrombin at +0.452 against +0.021.

Topomer fields

Cramer's topomer idea (topomer CoMFA, 2003), applied to fields. Cut the molecule at the bonds that attach its substituents to a common core and rebuild each substituent in a pose that is a **deterministic function of its topology**: canonical atom numbering, one embedding with a fixed seed, every rotatable torsion set to 180°, the attachment atom at the origin with its open valence along −x, and the remaining roll fixed by the principal axis of the heavy atoms. The same R-group then always lands in the same place, whatever molecule it came from — a test asserts the coordinates are identical, not merely close.

Two things make this fit CMF better than it fits CoMFA. There is no grid, so a fragment's field is just its atoms and a fragment–fragment kernel is an analytic number, which means the fragment library can be tabulated once and reused. And each attachment point keeps **its own region of space** (the substituent of position k is placed at $x = 100\ k$ Å, far enough that no Gaussian reaches the next site), so a molecule's kernel is exactly the sum of its per-position overlaps — which is what topomer CoMFA obtains by giving every position its own block of columns. A test checks that identity to 1e-9.

In the application it is *Tools → Replace molecules by their topomer substituents…*, and it is a **transformation of the data set** rather than a hidden switch: the molecules are replaced by their posed substituents, names, activities and the split are kept, and the data-set line says *[topomer substituents of …]*. That is the honest shape for it, because a topomer model's co-field and contributions live in the substituent frames and not around the original ligands. Reload the file to get the molecules back.

Choosing the core matters more than anything else. The maximum common substructure of a chemotype mixture is not a scaffold — on one benchmark set it came out as `CCO`, fused into every ring system, and cut nothing off at all. Give the core yourself as SMILES or SMARTS when you know it, and check the cuttable fraction the program reports: a core fused to the rest of the molecule cannot be removed without opening a ring, and what is left is not a substituent.

What the three routes are worth, measured

Every column below uses the same molecules, fields, α and λ grids and folds.

Collection	Series	Aligned q²	Topomer q²	The cut costs	The pose costs
Sutherland, curated frame	8	**+0.445**	+0.371	+0.058	+0.015
Mittal, automatic frame	31	+0.200	**+0.256**	−0.047	−0.009
Corrosion inhibitors, neutral	17	+0.248	+0.249	+0.061	−0.061
Corrosion inhibitors, 1 M HCl	17	+0.303	+0.292	+0.056	−0.045

The last two columns separate the two things the topomer construction does at once, by way of a control that keeps the same cut substituents at the same separated sites but leaves each one exactly where the alignment put it. The **pose rule is nearly free** — 0.015 q² against a careful superposition, and a *gain* of 0.045 to 0.061 where the frame is automatic, because a canonical pose is more consistent than an arbitrary embedded conformation. What costs something is the **cut**: separate sites discard the scaffold's own field and the geometry between one substituent and another, which is expensive when the arms' mutual placement is the pharmacophore (0.26 on

thermolysin, 0.18 on COX-2) and free or better than free when the scaffold is constant and the decorations small (BZR gains 0.123, DHFR 0.071).

Practical guidance. With a trustworthy curated alignment, keep it: the geometry is worth about 0.25 q^2 over counts. With an automatic alignment or none, prefer the frame-free routes — topomer for a congeneric series with a clear scaffold whose R-groups come off at acyclic single bonds, and the invariant kernel for anything pooled, multi-scaffold or ring-fused. Always report the count baseline beside the model, so that the value of the geometry is visible rather than assumed.

Heterogeneous series, step by step

When a set holds several chemotypes with no one common substructure, the *Tools* → *Heterogeneous sets* submenu is a four-step workflow:

1. **Find the chemotypes** — Bemis–Murcko frameworks, optionally generic, with small groups merged so that leave-one-chemotype-out does not degenerate into leave-one-molecule-out.
2. **Align within each chemotype** — a superposition that is meaningful, because the members of one cluster do share a substructure.
3. **Place the chemotypes in one frame** — each cluster positioned by maximising field similarity to the others, which is the best that can be done when no common substructure exists.
4. **Leave one chemotype out, with the alignment-free baseline** — the honest validation, reported beside the count kernel. Random folds flatter such sets by 0.11 to 0.47 q^2 on the five chemotype benchmarks here, which is why this step exists.

Protein pockets

Getting a pocket on screen

File → Fetch protein from the PDB... downloads an entry from RCSB and caches it under `~/.cmfqsar/pdb`, with the dialog pre-filled with the identifier the data set names. *File → Load protein structure...* reads a local PDB or mmCIF file instead. The reader is deliberately independent of RDKit, because crystal structures carry alternate locations, missing elements and non-standard residues that a chemistry toolkit would refuse to sanitise, and none of that matters for drawing.

The free-energy benchmark sets in `examples/openff` and `examples/schindler` give their ligands in the frame of a reference crystal structure and name it per record, so **nothing has to be superimposed**: opening such a set brings its pocket with it from `examples/pockets/`, which holds the residues within 8 Å of any ligand, taken whole, without waters and without the co-crystallised ligand. The *protein site* controls above the 3D view then draw the residues within a chosen distance as wire frame or sticks, with hydrogens, waters and residue labels optional.

Complementarity as numbers

A pocket is a set of atoms, so the same parameterisation that builds a ligand's fields builds the protein's, and the overlap of two Gaussian fields has the same closed form on either side. The *Complementarity* tab (CLI: `cmf_cli.py complement`) turns that into three answers:

- **How complementary a molecule is to the pocket** — overlap integrals between ligand and pocket fields, exact and without a grid, each divided by the two norms so that it is a cosine in $[-1, 1]$ and does not grow with molecular size. Complementarity is not similarity: the electrostatic term carries a minus sign, because a positive ligand region should meet a negative protein region; the hydrophobic term carries a plus; the hydrogen-bond terms pair the donors of one side with the acceptors of the other. Steric overlap is reported separately as *burial*, because bulk against the protein is contact and clash at once, with the number of clashing heavy-atom pairs beside it.
- **Where on the molecule it holds** — the same quantities on a smooth van der Waals surface give a value per vertex, drawn as a green-to-red surface, with a correlation per field over the surface. Electrostatics uses potentials rather than Gaussian densities here, because the potential is what the other side feels.
- **Whether the co-field agrees with the pocket** — the model's own map against the protein, by sign agreement, cosine or correlation. This is the strongest external check a CMF model can be given: the co-field was fitted to activities and never saw the protein.

Docking

Tools → Dock into the pocket... places the conformations of a molecule in the site by rigid-body optimisation. It is a convenience for getting a pose into the right frame, not a scoring function; the program's business is what happens after the poses exist.

Command-line reference

Everything the application does is available without it, which is what makes the program scriptable for a benchmark or a cluster run. The general form is

```
python cmf_cli.py <subcommand> <dataset> [options]
```

and `python cmf_cli.py <subcommand> --help` prints the full option list; what follows is the map, not every option.

Subcommand	What it does
`build`	build, cross-validate, test and save a model
`predict`	predict with a saved model, optionally aligning by confidence first
`hetero`	the heterogeneous-series workflow: cluster, align, place, leave-one-chemotype-out
`topomer`	model a series by its substituents, posed by rule
`map`	chemical-space map and activity landscape (kernel PCA + GTM)
`explain`	Shapley decomposition of one prediction, for any kernel
`mil`	multi-instance learning over conformations
`pharmacophore`	read a pharmacophore out of a fitted model's co-fields
`difference`	what one molecule's fields have that another's do not
`complement`	field complementarity with a protein pocket
`unalign`	give every molecule a random pose, the control experiment

The options of `build`, by group

Data. `--activity PROP`, `--activity-file TABLE`, `--test-file FILE`, `--test-fraction F`, `--ensemble-by KEY`.

Fields. `--fields electrostatic,steric,hydrophobic,hba,hbd[,prop:name]`, `--cif`, `--cif-scheme sybyl|element|element+aromatic`, `--cif-min N`, `--cif-types LIST`, `--charges gasteiger|mmff|file`, `--param rdkit|tripos|tripos-legacy`, `--prop-missing error|zero`.

Kernel. `--kernel fields|autocorrelation`, `--ac-sigma S`, `--ac-dmax D`, `--kernel-layer linear|gauss|arccos1|arccos2`, `--gammas LIST`, `--normalize`, `--raw-scale`, `--optimize-h`, `--mkl [align|alignf]`, `--mkl-rho R`.

Task and learner. `--task regression|classification|oneclass`, `--method krr|gp|svr|nusvr|svc|nusvc|gpc|ocsvm|ocmean|ocknn|ocgp|ockpca`, `--threshold T`, `--active-below`, `--class-property PROP`, `--oc-k K`, `--oc-components Q`, `--oc-lam L`, `--oc-score mean|mean_over_sd`, `--no-bias`.

Hyper-parameters. `--alphas LIST`, `--lams LIST`, `--Cs LIST`, `--epsilons LIST`, `--nus LIST`, `--lam-search grid|brent`, `--alpha-search grid|nelder-mead`, `--nm-start A,L`.

Conformations. `--conformations single|ensemble`, `--conformers N`, `--conformer-window A`, `--prune-rmsd A`, `--conformer-align self|core|reference|field`, `--conformer-align-ref I`,

`--conformer-weighting uniform|boltzmann|property`, `--kt KCAL`, `--max-conformers N`.

Validation. `--folds K`, `--cv-no-shuffle`, `--nested`, `--nested-on train|all`, `--outer-folds K`, `--repeats R`, `--seed S`.

Screening. `--decoys FILE`, `--align-decoys`, `--decoy-restarts N`, `--decoy-conformers N`.

Alignment. `--align-train-variance [MAX_RMSD]`, `--align-train-rounds N`, `--align-train-folds K`.

Output. `--save model.cmf`, `--predictions out.csv`, `--json out.json`.

Worked examples

```
# indicator fields, a test split and nested cross-validation
python cmf_cli.py build examples/benzamides_synthetic.sdf --activity pActivity --cif \
       --alphas 0.3,0.5,0.8,1.2 --lams 1e-3,1e-2,0.1,1 --test-fraction 0.25 \
       --nested --save model.cmf

# predict, with the new molecules aligned by prediction confidence first
python cmf_cli.py predict model.cmf new.sdf --align-variance 2.0 \
       --aligned-out aligned.sdf --out predictions.csv

# the published protocol of the 2013 paper on a Sutherland set
python cmf_cli.py build examples/sutherland/THR.sdf --charges file --param tripos-legacy \
       --raw-scale --cv-no-shuffle --lam-search brent --nested --nested-on all

# a frame-free model: the rotation-invariant autocorrelation kernel
python cmf_cli.py build series.sdf --kernel autocorrelation --ac-sigma 0.75 --ac-dmax 12

# a topomer model with the core given explicitly
python cmf_cli.py topomer examples/benzamides_synthetic.sdf --activity pActivity \
       --core "c1ccccc1C(=O)N" --alphas 0.3,0.5,1 --lams 1e-2,0.1,1

# one-class screening against a property-matched background
python cmf_cli.py build actives.sdf --task oneclass --method ockpca \
       --decoys background.sdf --align-decoys --decoy-conformers 5

# a classification model with the Gaussian-process classifier
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file --task classification \
       --threshold 6.84 --method gpc --Cs 1,10,100
```

Using it as a library

The library is the layer the application and the command-line tool are built on, and it is small enough to use directly in a notebook.

```
from cmfqsar import FieldSet, CMFModel, load_dataset
from cmfqsar.fields import AtomTable, build_fields, cif_types_in
from cmfqsar.model import grid_search

ds = load_dataset("aligned.sdf", activity_prop="pIC50")
fs = FieldSet.create(cif_types=cif_types_in(ds.mols, "sybyl", 5))
mf = [build_fields(m, fs, n) for m, n in zip(ds.mols, ds.names)]
train = AtomTable.from_molecules(mf)
model = CMFModel(fieldset=fs, method="krr")
grid_search(train, ds.activity, model, alphas=[0.3, 0.5, 1.0], lams=[1e-3, 1e-2, 0.1, 1])
model.fit(train, ds.activity, cv_folds=10, reuse_kernels=True)
print(model.stats.summary())
```

Decomposing one prediction, which works for the nonlinear kernels as well:

```
from cmfqsar.shapley import atom_shapley, field_shapley, scaffold_decomposition

res = atom_shapley(model, mf[0])                       # one value per atom
res = field_shapley(model, mf[0], interactions=True)   # one value per field type
res = scaffold_decomposition(model, mf[0], ds.mols[0], ds.mols[1:20])   # per substituent
print(res.prediction, res.baseline, res.table(), res.top_interactions(5))
```

The modules, one line each:

```
cmfqsar/fields.py       field parameterisation; FieldSet / MoleculeFields / AtomTable
cmfqsar/kernels.py      analytic field kernels, vectorised
cmfqsar/model.py        KRR / SVR / GP, cross-validation, grid search, nested CV, statistics
cmfqsar/cofields.py     co-fields and molecular fields on grids, isosurfaces, cube export
cmfqsar/dataio.py       reading SDF / MOL2 / SMILES, model persistence, exports
cmfqsar/align.py        3D embedding, MCS alignment, field-similarity refinement, unalign
cmfqsar/varalign.py     alignment by prediction confidence (GP posterior variance)
cmfqsar/invariant.py    rotation-invariant field autocorrelation kernels
cmfqsar/topomer.py      topomer fields: substituents posed by rule
cmfqsar/chemotypes.py   scaffold clusters, placement, grouped folds, the count kernel
cmfqsar/pharmacophore.py  typed features with tolerance spheres, from the co-field
cmfqsar/fielddiff.py    field differences against a reference molecule
cmfqsar/shapley.py      Shapley decomposition over atoms, substituents, field types
cmfqsar/gpc.py          Laplace Gaussian-process classification
cmfqsar/oneclass.py     centroid, k-NN, GP and kernel-PCA one-class descriptions
cmfqsar/conformers.py   conformational ensembles: generation, weighting, grouping
cmfqsar/mil.py          multi-instance learning over conformations
cmfqsar/decoys.py       posing a background set for screening evaluation
cmfqsar/gtm.py          chemical-space maps and activity landscapes
cmfqsar/protein.py      PDB / mmCIF reading, binding-site selection, frame check
cmfqsar/complementarity.py  pocket fields, complementarity scores, co-field agreement
cmfqsar/docking.py      rigid-body placement of conformations in the pocket
cmfqsar/gui/            the PySide6 application
```

The example data sets

Set	Contents
`examples/benzamides_synthetic.sdf`	109 aligned N-phenylbenzamides with a *synthetic* activity: the overlap of their fields with a hidden pseudo-receptor co-field plus noise (σ = 0.25). A model has to recover those zones; $q^2 \approx 0.87$ against a noise ceiling of 0.94. The set to learn the program on, because the right answer is known.
`examples/steroids_cbg.sdf`	the classic CoMFA benchmark: 31 steroids (Cramer, Patterson & Bunce 1988) with corticosteroid-binding-globulin affinities, in the corrected Gasteiger-group geometries, re-superimposed on the steroid skeleton of testosterone. Customary split 1–21 / 22–31, leave-one-out $q^2 \approx 0.79$.
`examples/sutherland/*.sdf`	the eight literature sets of the 2013 paper (Sutherland, O'Brien & Weaver 2004): ACE, AChE, BZR, COX-2, DHFR, GPB, THER, THR, with the aligned MMFF94s conformations as distributed, the published split, and the original Sybyl types and MNDO ESP charges as per-atom properties. Choose *Partial charges: file*.
`examples/mittal/*.sdf`	38 of the 40 benchmark sets of Mittal, McKinnon & Sorich (2009), built here from their 2D supplementary data by ETKDG and an MCS superposition, with each molecule's fit quality recorded in `align_rmsd`. The collection to use when you want to know how a method behaves on an *automatic* alignment.
`examples/openff/*.sdf`	21 congeneric series, 513 ligands, from the Open Force Field protein–ligand benchmark, each posed in its target's crystal structure; activity tag `pAffinity`.
`examples/pockets/*.pdb`	the binding sites of those series, trimmed and already in the ligands' frame, so they open together with the data.
`examples/thrombin_atom_properties.sdf`	a worked example of `prop:<name>` fields: per-atom quantum properties carried in an SDF.

`docs/benchmark.md` compares the program with Table 2 of the 2013 paper on the eight literature sets. `benchmarks/run_sutherland.py` reruns that comparison, `benchmarks/run_topomer.py` measures the alignment against the topomer rule (`--mittal` for the automatically aligned collection), `benchmarks/run_alignment.py` prices a curated alignment against an automatic one, and `benchmarks/run_corrosion.py` does the same for corrosion-inhibitor series.

When something goes wrong

The 3D view is black, or the program will not start. The viewer needs working OpenGL through PyOpenGL and pyqtgraph. On a remote desktop or in a virtual machine, software rendering is usually the fix (on Windows, a Mesa `opengl32.dll` beside the program; on Linux, `LIBGL_ALWAYS_SOFTWARE=1`). Everything except the 3D view works regardless, and the command-line tool needs no graphics at all.

q² is high but the test predictions are poor. Three usual causes, in order of frequency: the hyper-parameters were chosen on the same cross-validation you are quoting (run *External CV*); the set holds several chemotypes and random folds are flattering it (use leave-one-chemotype-out); or the alignment is doing work that will not transfer (compare against the count baseline).

q² is negative. The model predicts worse than the mean: check that the activity column is the right one and on a logarithmic scale, and that the molecules really share a frame. For R^2_{pred} on a narrow test set it may be an artefact of the variance convention, and the program reports both.

The optimum sits at the edge of the α or λ grid. Extend the grid. The *Hyper-parameter search* tab exists to make this visible.

A per-atom property is rejected. The hydrogen count almost certainly changed when the file was loaded. Write the property for all atoms including hydrogens, or pass `--prop-missing zero` deliberately.

The topomer transformation drops most of the molecules. The core is fused to the rest of them. Look at the cuttable fraction in the log and give a core joined to its R-groups by acyclic single bonds.

A decoy set gives suspiciously good enrichment. Check that the decoys are property-matched and posed in the training frame; cross-target decoys are separable by almost anything.

Co-fields are unavailable. A nonlinear kernel layer or the autocorrelation kernel source is selected, and neither is linear in the field, so the co-field does not exist. Use the *Shapley* tab instead.

It is slow. Kernel cost grows with the square of the total atom count, so the expensive sets are those with many large molecules. Cut the α grid first, since each α needs its own kernel, and remember that the count kernel is 10 to 34 times faster than the Gaussian route for a baseline.

Appendix A — Field parameterisation

Two parameterisations are available (*Increments* in the model panel, `--param`):

Field	`rdkit`, the default	`tripos`, the original CMF scripts
electrostatic	partial charge: Gasteiger, MMFF94, or read from the file	the same
steric	Tripos vdW well depth and width α/R^2_vdW, parameters by element	parameters by Sybyl atom type, Tripos 5.2 table
hydrophobic	Wildman–Crippen atomic logP contribution	in-house one-atom fragment increments by Sybyl type and attached hydrogens
H-bond acceptor / donor	approximate Abraham-type increments, donor weight on the polar hydrogen	in-house increments from Abraham's A and B constants, donor weight on the heavy atom
indicator field	1 for atoms of the given type	the same
`prop:<name>`	the value of a per-atom property carried by the file	the same

`tripos-legacy` reproduces the hydrogen-counting bug of the original R scripts, which is needed to match the published numbers exactly and should not be used for new work.

Appendix B — Files and tags the program understands

Tag or convention	Meaning
any numeric SD property	candidate activity; the loader guesses and the dialog confirms
`subset`, `set`, `split`	the train/test split, as `train`/`test` or 1/2
`atom.dprop.<name>`	a per-atom double property, written and read by RDKit
`atom.prop.sybyl_type`	per-atom Sybyl types, used automatically for indicator fields
a molecule property with one number per atom	the same per-atom property in quantum-chemistry dump form
`.cmf`	a saved model (joblib): field set, kernels, coefficients, training molecules, settings
Gaussian cube	the export format for co-fields, molecular fields, Shapley fields and difference maps
`REMARK 400` in a pocket PDB	the provenance of a trimmed binding site

Appendix C — The papers

The method. Zhokhova, Baskin, Bakhronov, Palyulin & Zefirov, *Method of continuous molecular fields in the search for quantitative structure–activity relationships*, Dokl. Chem. **429** (2009)

1 · Baskin & Zhokhova, *The continuous molecular fields approach to building 3D-QSAR models*,

J. Comput.-Aided Mol. Des. **27** (2013) 427. · Baskin & Zhokhova, in *Application of Computational Techniques in Pharmacy and Medicine*, Springer (2014) 433. · Sitnikov, Zhokhova, Ustynyuk, Varnek & Baskin, *Continuous indicator fields: a novel universal type of molecular fields*, J. Comput.-Aided Mol. Des. **29** (2015) 233. · Baskin & Zhokhova, *Descriptors based on continuous indicator fields for 3D-QSAR studies*, MOL2NET (2017). · Baskin & Zhokhova, *Continuous molecular fields and the concept of molecular co-fields in structure–activity studies*, Future Med. Chem. **11** (2019) 2701.

One-class classification and virtual screening. Karpov, Baskin, Zhokhova & Zefirov, *Method of continuous molecular fields in the one-class classification task*, Dokl. Chem. **440** (2011) 263. · Karpov, Baskin, Zhokhova, Nawrozkij, Zefirov, Yablokov, Novakov & Zefirov, *One-class approach: models for virtual screening of non-nucleoside HIV-1 reverse transcriptase inhibitors based on the concept of continuous molecular fields*, Russ. Chem. Bull. **60** (2011) 2418.

Methods the program borrows. Cramer, Patterson & Bunce, *Comparative molecular field analysis (CoMFA)*, J. Am. Chem. Soc. **110** (1988) 5959. · Cramer, *Topomer CoMFA*, J. Med. Chem. **46** (2003) 374. · Bemis & Murcko, *The properties of known drugs: molecular frameworks*, J. Med. Chem. **39** (1996) 2887. · Hopfinger, Wang, Tokarski, Jin, Albuquerque, Madhav & Duraiswami, *4D-QSAR*, J. Am. Chem. Soc. **119** (1997) 10509. · Bishop, Svensén & Williams, *GTM: the generative topographic mapping*, Neural Comput. **10** (1998) 215. · Cho & Saul, *Kernel methods for deep learning*, NIPS (2009). · Rasmussen & Williams, *Gaussian Processes for Machine Learning*, MIT Press (2006). · Truchon & Bayly, *Evaluating virtual screening methods*, J. Chem. Inf. Model. **47** (2007) 488. · Sutherland, O'Brien & Weaver, *A comparison of methods for modeling quantitative structure–activity relationships*, J. Med. Chem. **47** (2004) 5541. · Mittal, McKinnon & Sorich, *Comparison data sets for benchmarking QSAR methodologies in lead optimization*, J. Chem. Inf. Model. **49** (2009) 1810.